Day 34 Practice Problems: Terminal & Shell Operations

This suite contains three production-grade engineering challenges designed to solidify your mastery of command-line operations, path navigation, file lifecycle management, and defensive system administration taught in Day 34.

Challenge 1: Production Server Triage & Path Resolution

Background & Context

You have logged into an Ubuntu GPU compute instance (`ai-prod-node-04`) via SSH. A computer vision training job crashed at 02:30 AM due to a missing dataset pointer. Your task is to inspect the filesystem, locate error logs, verify environment variables, and inspect hidden dotfiles without GUI access.

Requirements & Tasks

1. **Print Current Location**: Confirm your initial landing directory using the POSIX command to print the working directory.
2. **Detailed Directory Listing**: List all contents in your current folder including hidden dotfiles (e.g., `.bashrc` , `.env` , `.git`), file sizes, permissions, and modification timestamps.
3. **Absolute vs Relative Navigation**:
    - The production logs are located at `/var/log/cv_pipeline/crash_dump.log` . Describe whether this is an absolute or relative path and why.
    - Navigate into `/var/log/cv_pipeline/` using an absolute path.
    - From `/var/log/cv_pipeline/` , navigate up two directory levels to `/var/` using a single relative command without specifying `/var` explicitly.
4. **Home Directory Return**: Return directly to your user home directory (`/home/engineer/`) in one single command using the shell shortcut.

Challenge 2: MLOps Project Workspace Scaffolding

Background & Context

You are initializing a standardized directory structure for a new distributed NLP sentiment classifier project named `sentiment_llm`. Rather than clicking through file explorers, you must scaffold the entire project hierarchy using Bash commands.

Requirements & Tasks

1. **Directory Tree Creation**:
   - Create the root project directory `sentiment_llm` and enter it.
   - Create three distinct subdirectories: `data`, `src`, and `checkpoints`.
   - Inside `data`, create two separate subdirectories: `raw` and `processed`.
2. **File Initialization**:
   - Inside `src`, initialize three empty files: `dataset.py`, `model.py`, and `train.py`.
   - In the project root (`sentiment_llm`), create the project manifest `requirements.txt` and a hidden environment file `.env`.
3. **Timestamp Refresh**:
   - Suppose `requirements.txt` was created 10 minutes ago. Use the POSIX touch utility to update its modification timestamp to the current system time without modifying its contents or creating a new file.
4. **Validation**:
   - Run a single command with flags to verify that all files (including hidden `.env`) and directories exist with readable timestamps.

Challenge 3: Defensive Disk Cleanup & Safe Deletion Governance

Background & Context

The scratch storage volume mounted at `/mnt/scratch_storage` is 98% full. Old training runs generated thousands of temporary epoch checkpoints and temporary lock files. You must safely reclaim space without accidentally wiping production models or critical operating system paths.

Requirements & Tasks

1. **Empty vs Non-Empty Directory Deletion**:
   - You find an obsolete empty folder named `stale_cache/`. Which command safely deletes it while ensuring the operation aborts if the folder is accidentally non-empty?
   - You find an old experiment run directory `run_epoch_01/` containing 50 model weights (`.pt`) and log files (`.csv`). Which command removes this entire non-empty directory tree?
2. **Risk Analysis of Dangerous Patterns**:

- Explain what happens if an engineer accidentally types:

  ```
  rm -rf / tmp/old_checkpoints
  ```

  (Notice the space between `/` and `tmp/old_checkpoints`). Explain the catastrophic consequences and system impact.

- Why is file deletion in the terminal considered irreversible compared to deleting files via a graphical Desktop GUI (e.g., Windows Recycle Bin or macOS Trash)?

3. **Safe Deletion Protocol**:
   - Outline a 3-step defensive workflow before executing any recursive `rm` command on production machines.

Solutions Reference

For complete step-by-step commands and architectural explanations, inspect Solutions.md.\n